NETWORKING FUNDAMENTALS

IPv4 Addressing and Subnetting

A Beginner's Guide for Amazon Virtual Private Cloud

A step-by-step introduction to address ranges, CIDR, subnet calculations, and VPC design

1 Introduction: Why IP Addressing Matters

A computer network is a group of connected devices that can exchange information. To deliver information to the right device, a network needs addresses. An **IP address** is a numerical address used to identify an interface on an Internet Protocol network and to help route traffic toward it.

Think of an IP address like a postal address: it helps direct a message to a destination. The analogy has limits. IP addresses identify network interfaces rather than people or permanent physical locations, and routers use structured prefixes and routing rules rather than reading a street address.

Large networks are divided into smaller networks so addresses can be organized, routes managed, and workloads separated. Amazon Virtual Private Cloud (Amazon VPC) is an isolated virtual network in AWS. Its IPv4 range and the smaller ranges assigned to its subnets are foundational design choices.

After this guide, you should be able to read an IPv4 address, recognize historical classes and private ranges, interpret CIDR notation, calculate subnet boundaries and sizes, and reason about address planning for a VPC.

2 Understanding IPv4 Addresses

IPv4 means Internet Protocol version 4. An IPv4 address is a 32-bit value, usually written as four decimal numbers separated by dots. Each number is an **octet**: a group of eight bits. An octet has $2^8 = 256$ possible bit patterns, so its decimal value can range from 0 through 255.

For example, `192.168.1.10` has four octets:

First octet	Second octet	Third octet	Fourth octet	Address
192	168	1	10	`192.168.1.10`

Computers represent each octet in binary, a number system that uses only 0 and 1. The decimal and binary forms of the four octets are:

$$\begin{aligned} 192 &= \texttt{11000000} \\ 168 &= \texttt{10101000} \\ 1 &= \texttt{00000001} \\ 10 &= \texttt{00001010} \end{aligned}$$

Grouped as a complete 32-bit address, this is

`11000000.10101000.00000001.00001010`.

The dotted decimal form is compact and easier for people to read. The binary form makes the address's bit structure visible, which is useful when defining network boundaries.

There are $2^{32} = 4{,}294{,}967{,}296$ possible 32-bit patterns. This is the size of the IPv4 address space, not the number available for ordinary host assignment: some blocks are reserved for private networks, multicast, documentation, and other special purposes.

3 Historical IPv4 Address Classes

Before CIDR, IPv4 networks were commonly allocated using a **classful** system. The first bits of the first octet identified a class; the class implied a default network size. This history helps explain familiar ranges, but modern VPC design uses CIDR prefixes rather than class rules.

Class	First-octet range	Leading bits	Historical purpose and ordinary unicast host use?
A	0–127 by pattern (ordinary: 1–126)	`0xxxxxxx`	Large unicast networks; yes, except special-purpose values.
B	128–191	`10xxxxxx`	Medium-sized unicast networks; yes.
C	192–223	`110xxxxx`	Smaller unicast networks; yes.
D	224–239	`1110xxxx`	Multicast groups; no.
E	240–255	`1111xxxx`	Experimental or future use; no.

The leading-bit patterns create the boundaries: the remaining bits in the first octet vary within each pattern. Class A values 0 and 127 have special purposes, so traditional ordinary Class A assignments are generally described as first-octet values 1–126. Historically, the default network prefixes were Class A `/8`, Class B `/16`, and Class C `/24`.

Those defaults are not mandatory modern rules. **Classless Inter-Domain Routing (CIDR)** allows a network to use a prefix length chosen for its needs. CIDR is the system used for modern network planning, including AWS VPCs.

Address example	Historical class	Note
`10.4.2.9`	A	First octet 10 is in 1–126.
`172.20.5.8`	B	First octet 172 is in 128–191; it is also RFC 1918 private space.
`172.50.5.8`	B	Historically Class B, but not RFC 1918 private space.
`192.168.1.10`	C	First octet 192 is in 192–223.
`230.1.2.3`	D	Class D is for multicast.
`250.1.2.3`	E	Class E is not ordinary unicast host space.

Common misconception: An address beginning with 172 is not automatically private. Only `172.16.0.0/12` within that historical Class B range is RFC 1918 private space.

4 Public and Private IPv4 Addresses

Private IPv4 ranges exist so organizations can number internal networks without obtaining a globally unique address for every device. Different, unconnected networks can reuse these ranges. Private addresses are not globally routed across the public internet. A **publicly routable** address, by contrast, can be routed on the public internet when routing and policy permit. Being outside a private range does not by itself guarantee public routability; other special-purpose blocks exist.

RFC 1918 defines three private ranges [1]:

CIDR block	Inclusive range	Useful way to read it
`10.0.0.0/8`	`10.0.0.0`–`10.255.255.255`	Every address whose first octet is 10.
`172.16.0.0/12`	`172.16.0.0`–`172.31.255.255`	Second octet is 16 through 31 when the first octet is 172.
`192.168.0.0/16`	`192.168.0.0`–`192.168.255.255`	First two octets are 192.168.

The middle range is often misread. In `172.16.0.0/12`, the first 12 bits are fixed: all 8 bits of the first octet and the first 4 bits of the second octet. Since 172 is `10101100`, the first octet is fixed. The first 4 bits of the second octet are `0001`; its remaining 4 bits can vary:

$$\underbrace{0001}_{\text{fixed}}\,\underbrace{0000}_{\text{smallest}} = 16 \qquad \text{through} \qquad \underbrace{0001}_{\text{fixed}}\,\underbrace{1111}_{\text{largest}} = 31.$$

Thus the second octet ranges from 16 to 31, not 0 to 255. The address `172.20.0.1` is inside this private range because 20 is between 16 and 31. The address `172.50.0.1` is outside it because 50 is not.

Other special-purpose IPv4 blocks include loopback addresses such as `127.0.0.1`, link-local addresses in `169.254.0.0/16`, documentation ranges, and multicast space. Therefore, "not RFC 1918" and "usable public address" are not equivalent statements.

Private addressing is not a security control by itself. Routing, security groups, network ACLs, and other controls determine which traffic can pass. In AWS, private ranges are commonly used when selecting VPC CIDRs. Choose ranges that will not overlap with networks the VPC may later connect to: overlapping CIDRs can make routing ambiguous or prevent connectivity over VPC peering, VPN, or similar links.

5 Binary Basics Needed for CIDR

Binary is a base-2 number system: each bit is either 0 or 1. Each IPv4 octet contains eight bit positions, with these place values:

Bit position	1	2	3	4	5	6	7	8
Place value	128	64	32	16	8	4	2	1

To convert binary to decimal, add the place values that have a 1. For example,

$$\texttt{11001010} = 128 + 64 + 8 + 2 = 202.$$

To write 13 in binary, select place values that add to 13: $8 + 4 + 1 = 13$. Put 1s in those places and 0s elsewhere, giving `00001101`.

An IPv4 address has four octets, or 32 bits total. A CIDR prefix such as `/16` says that the first 16 bits are the network prefix; the other bits vary within that address block. This division between fixed and varying bits is the basis of subnetting.

6 CIDR Notation Explained

CIDR writes an address block as `IP-address/prefix-length`. In `192.168.1.0/24`, the `/24` means that the first 24 of the 32 bits identify the network prefix. The remaining bits identify positions within that block.

The two essential CIDR relationships

$$\text{network bits} + \text{remaining address bits} = 32,$$
$$\text{remaining bits} = 32 - \text{prefix length}.$$

The remaining bits determine the number of bit patterns, and therefore the total number of addresses, in the block:

$$\text{total addresses} = 2^{\text{remaining bits}} = 2^{32-\text{prefix length}}.$$

Prefix	Remaining bits	Total addresses	Example interpretation
/8	24	16,777,216	24 bits can vary.
/16	16	65,536	16 bits can vary.
/20	12	4,096	12 bits can vary.
/24	8	256	One full octet can vary.
/26	6	64	6 bits can vary.
/28	4	16	4 bits can vary.
/30	2	4	2 bits can vary.
/32	0	1	One exact IPv4 address.

A larger prefix number leaves fewer bits to vary and therefore describes a smaller block: a /16 contains more addresses than a /24, and a /24 contains more than a /28. A /0 has no fixed prefix bits and mathematically covers all 2^{32} IPv4 bit patterns. In routing, 0.0.0.0/0 is commonly used as a default route; it does not mean every address is appropriate for assignment to one subnet.

The prefix length is part of CIDR notation. It is not the historical address class. For example, a first octet in the old Class A range can be written with prefixes other than /8.

7 Network Address, Host Addresses, and Broadcast Address

For an ordinary IPv4 subnet such as 192.168.1.0/24, the **network address** identifies the block, while addresses inside it can identify hosts. In a traditional broadcast-capable IPv4 subnet, the last address is the **broadcast address**, used to send a packet to all hosts on that subnet.

Meaning	Address in 192.168.1.0/24
Network address	192.168.1.0
First conventional usable host	192.168.1.1
Last conventional usable host	192.168.1.254
Broadcast address	192.168.1.255
Total addresses	$2^8 = 256$
Conventional usable host addresses	$256 - 2 = 254$

For ordinary subnet arithmetic, the first address is the network identifier and the last is the broadcast address, leaving the addresses between them for hosts. Prefixes /31 and /32 are exceptions: a /31 is used for point-to-point links under special rules, and

a /32 denotes one address or host route. They do not follow the usual "subtract two" host-count rule [2].

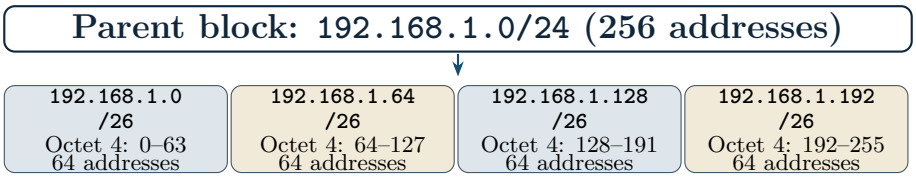
AWS reserves five IPv4 addresses in an ordinary subnet (the first four and last). A /24 therefore has 251 AWS-assignable addresses, not the conventional 254; AWS does not support subnet broadcast [3].

8 What Is Subnetting and Why Is It Important?

Subnetting means dividing a larger IP address block into smaller address blocks. A smaller block is called a subnet. Subnetting lets a network planner assign address space in manageable pieces instead of treating one large range as a single pool.

Subnetting is useful for organizing resources, separating workloads, planning address allocation, managing routes, and supporting a security architecture. In AWS, VPC CIDR space is divided into subnets that can serve different workloads and routing purposes. Subnetting alone does not secure resources; access still depends on routing and security controls.

The diagram shows how a parent /24 is divided into four equal child blocks. Each child has $2^{32-26} = 64$ addresses, and the four ranges together contain all 256 addresses of the parent.



Four equal, aligned, non-overlapping /26 blocks

Child subnets must be correctly aligned within the parent and must not overlap. Creating a subnet does not automatically make it public or private. In AWS, routing configuration determines a subnet's routing behavior; its name or CIDR alone does not.

9 The Block Size Method

The **block size** is the number of consecutive addresses in the octet where subnet boundaries advance, when using the changing-octet method. First write the subnet mask in dotted-decimal form and identify the octet with a partial mask value (a value between 1 and 254). In that octet:

$$\text{block size} = 256 - \text{subnet-mask value in the changing octet.}$$

The partial mask fixes some bits in that octet and leaves the rest to vary; the subtraction gives the number of addresses between successive boundaries. This

shortcut applies to that changing octet for standard IPv4 CIDR subnet boundaries. It is not a rule to subtract any arbitrary mask octet from 256 without first identifying the correct octet. If the mask octet is 0, the block size in that octet is 256; if it is 255, that octet does not change.

9.1 Example A: A /24 block

The mask for `192.168.1.0/24` is `255.255.255.0`. The first three octets are fixed; the fourth octet is the changing octet. Its mask value is 0, so the block size there is $256 - 0 = 256$. The fourth octet can take all values 0–255, giving $2^8 = 256$ addresses in this `/24`.

9.2 Example B: Dividing a /24 into /26 blocks

The mask for `192.168.1.0/26` is `255.255.255.192`. The fourth octet is the changing octet, and

$$256 - 192 = 64.$$

The binary mask value 192 is `11000000`, leaving six low bits free; $2^6 = 64$ possible values, which explains the block size. Therefore subnet starts advance by 64: 0, 64, 128, and 192. Each block ends one address before the next start.

Block	Address range	Addresses
`192.168.1.0/26`	`192.168.1.0`–`192.168.1.63`	64
`192.168.1.64/26`	`192.168.1.64`–`192.168.1.127`	64
`192.168.1.128/26`	`192.168.1.128`–`192.168.1.191`	64
`192.168.1.192/26`	`192.168.1.192`–`192.168.1.255`	64

These four consecutive blocks partition the original `192.168.1.0/24` with no gaps or overlap.

9.3 Example C: /27 boundaries

The mask for `/27` is `255.255.255.224`; the changing-octet calculation is $256-224 = 32$. Within `192.168.1.0/24`, the valid starts in the fourth octet are

$$0,\ 32,\ 64,\ 96,\ 128,\ 160,\ 192,\ 224.$$

Eight `/27` blocks fit in one `/24` because each has 32 addresses and $8 \times 32 = 256$.

9.4 Example D: /28 boundaries

The mask for `/28` is `255.255.255.240`, so the block size is $256 - 240 = 16$. The first boundaries are 0, 16, 32, 48, and 64; continuing by 16 reaches 240. Thus 16 blocks of 16 addresses each fit in one `/24` ($16 \times 16 = 256$).

9.5 Example E: A /20 block

For 172.16.0.0/20, the mask is 255.255.240.0. The third octet is changing, because its mask value is 240. The block size is $256 - 240 = 16$, so boundaries in that octet are 0, 16, 32, 48, 64, and onward in steps of 16. A /20 has 12 remaining bits, or $2^{12} = 4096$ addresses.

The subnet 172.16.48.0/20 starts at the third-octet boundary 48. Sixteen values fit in that third-octet block, so it covers third-octet values 48 through 63. The fourth octet then spans 0 through 255:

172.16.48.0 through 172.16.63.255.

The changing octet depends on the prefix: it is the fourth octet in the /24, /26, /27, and /28 examples, but the third octet in the /20 example.

10 How to Find the Next Subnet

To find the next equal-sized subnet, follow these steps:

1. Read the CIDR prefix and write its subnet mask.
2. Identify the changing octet: the octet with a partial mask value, or the next octet when the mask ends at an octet boundary.
3. Calculate the block size in that octet.
4. Add the block size to the current subnet's starting value in that octet.
5. Check the full address range and confirm it remains inside the parent CIDR block.

Example 1: Next /26 subnet

For 192.168.1.0/26, the changing octet is the fourth and the block size is $256 - 192 = 64$. Add 64 to the starting fourth octet: the next network is 192.168.1.64/26. Adding 64 again gives 192.168.1.128/26, then 192.168.1.192/26. The next start would be 256, beyond this fourth-octet range; the four listed blocks exactly fill 192.168.1.0/24.

Example 2: Next subnet after 192.168.1.64/27

For a /27, the changing octet is the fourth and the block size is $256 - 224 = 32$. Add 32 to 64: $64 + 32 = 96$. The next subnet is 192.168.1.96/27, which spans 192.168.1.96–192.168.1.127.

Example 3: Next /20 subnet

Within the larger 172.16.0.0/16 range, 172.16.32.0/20 has a block size of 16 in the third octet. Add 16 to 32: the next subnet is 172.16.48.0/20.

The block size must be added to the changing octet, not automatically to the fourth octet. For example, adding 16 to the fourth octet when working with a /20 gives the

wrong boundary. A valid subnet network address must be aligned to a block boundary. `192.168.1.64/26` is aligned because 64 is a multiple of 64; `192.168.1.50/26` is not. The latter address is within the `192.168.1.0/26` block, but it is not that block's network address.

11 How Many Subnets Can Be Created?

When a parent CIDR block is divided into equal-sized, smaller blocks, the **original prefix** describes the parent, the **new prefix** describes each child, and their difference is the number of **borrowed bits**. Each borrowed bit doubles the number of possible child networks:

Equal-sized subnet count

$$\text{borrowed bits} = \text{new prefix} - \text{original prefix},$$
$$\text{number of subnets} = 2^{\text{borrowed bits}}.$$

This formula applies when a parent is divided into equal-sized subnets using a longer prefix.

Example A: `/24` divided into `/26`

Borrowed bits: $26 - 24 = 2$. Number of child subnets: $2^2 = 4$. Each child has $2^{32-26} = 64$ addresses, and $4 \times 64 = 256$, the parent block's total.

Example B: `/24` divided into `/27`

Borrowed bits: $27 - 24 = 3$. Number of child subnets: $2^3 = 8$. Each child has 32 addresses, so $8 \times 32 = 256$.

Example C: `/16` divided into `/20`

Borrowed bits: $20 - 16 = 4$. Number of child subnets: $2^4 = 16$. Each `/20` has $2^{12} = 4096$ addresses, so the 16 children contain $16 \times 4096 = 65{,}536$, the total in a `/16`.

Subnet count and addresses per subnet are different questions. Within one `/24`:

Child prefix	Equal-sized child subnets	Addresses in each child
`/25`	2	128
`/26`	4	64
`/27`	8	32
`/28`	16	16

12 How Many Addresses and Usable Hosts?

The prefix determines the total number of addresses, whether or not all of them can be assigned to hosts:

$$\text{total IPv4 addresses} = 2^{32-\text{prefix length}}.$$

For traditional IPv4 subnets with prefixes from `/0` through `/30` where network and broadcast addresses have their usual meanings, conventional host arithmetic generally subtracts those two addresses. In practice, `/0` is commonly a route covering the IPv4 space, not a single assignable host subnet. The `/31` point-to-point case and `/32` host-route case are exceptions.

Prefix	Total addresses	Conventional usable host interpretation
`/24`	256	254 (256 − 2)
`/25`	128	126 (128 − 2)
`/26`	64	62 (64 − 2)
`/27`	32	30 (32 − 2)
`/28`	16	14 (16 − 2)
`/29`	8	6 (8 − 2)
`/30`	4	2 (4 − 2)
`/31`	2	Special point-to-point use; both addresses may be used
`/32`	1	One address; commonly a host route

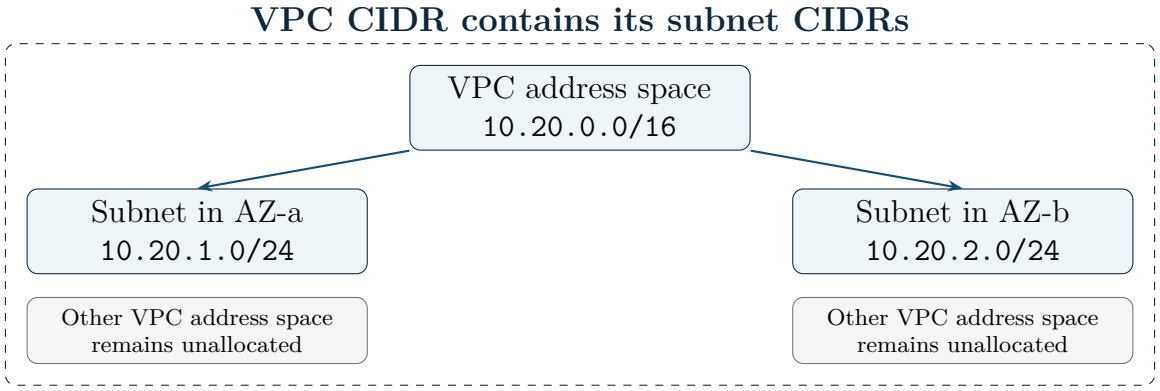
Keep these four quantities separate:

1. **Total addresses** are all bit patterns in the CIDR block.
2. **Conventional usable hosts** usually subtract the network and broadcast addresses for traditional subnets.
3. **AWS-reserved addresses** are addresses AWS does not make available for ordinary assignment in a subnet.
4. **Actually assignable addresses** depend on platform reservations and applicable network rules.

For an ordinary AWS IPv4 subnet, AWS reserves five addresses: the first four and the last address in the subnet. Therefore, a `/27` has $2^{32-27} = 32$ total addresses and ordinarily provides $32 - 5 = 27$ addresses available for assignment. A `/24` has 256 total and ordinarily 251 assignable. These counts are not the traditional "total minus two" convention. AWS does not support subnet broadcast, even though the final address is reserved under its rules. AWS subnet sizes and reserved-address behavior are service-specific; confirm the applicable rules when using special address arrangements [3].

13 Applying CIDR and Subnetting to Amazon VPC

An Amazon VPC is a logically isolated virtual network. A VPC has an IPv4 CIDR block, and its subnets use smaller CIDR blocks within that address space. Each subnet belongs to one Availability Zone. Multiple subnet ranges in the same VPC must not overlap [3].



The diagram is conceptual: the two `/24` subnets are separate parts of the parent `/16`; they do not need to consume all of it. The unused space leaves room for additional subnets and growth.

Each subnet is associated with a route table, which determines where traffic is sent. In AWS, a subnet with a route to an Internet Gateway is called a public subnet. However, an instance generally also needs a public IPv4 or IPv6 address and appropriate security configuration to communicate directly with the internet. A private subnet has no direct route to an Internet Gateway; it may still use other connectivity, such as a NAT Gateway for outbound IPv4 access [4].

A subnet is not public merely because it uses an RFC 1918 address, has the word "public" in its name, or belongs to a VPC. Routing determines the subnet's path; security groups, network ACLs, and other controls affect what traffic is allowed. Subnetting supports network organization but is not by itself sufficient protection for workloads.

VPC CIDR planning must account for address capacity and future connectivity. A range that seems large enough today can create difficulty if it overlaps an office network, another VPC, or a partner network that may later connect through peering or VPN.

14 A Practical CIDR Calculation Checklist

Use this checklist for a CIDR problem or an initial subnet plan:

1. **Read the prefix.** It tells you how many leading bits are fixed.

2. **Calculate remaining bits.** Subtract the prefix from 32 to find the variable bits.
3. **Calculate total addresses.** Raise 2 to the number of remaining bits.
4. **Find the subnet mask.** Convert the prefix's run of 1 bits into dotted-decimal octets.
5. **Identify the changing octet.** Find the partial-mask octet (or the next octet for a boundary prefix).
6. **Calculate block size where applicable.** Use 256− the mask value in the changing octet.
7. **Find valid boundaries.** Count multiples of the block size and check alignment.
8. **Count equal-sized subnets if there is a parent.** Use $2^{\text{new prefix}-\text{parent prefix}}$.
9. **Calculate conventional host counts where relevant.** Usually subtract network and broadcast addresses; check `/31` and `/32` exceptions.
10. **Account for AWS reservations.** For an ordinary AWS IPv4 subnet, subtract the five reserved addresses.
11. **Check parent membership and overlap.** Every child must fit within the parent and must not overlap another child subnet.

15 Worked Problems and Practice Questions

Try each problem before reading its answer. The set begins with recognition and advances to combined calculations.

1. Classify by historical class: `10.1.1.1`, `172.20.0.1`, and `192.168.0.1`. Then classify `230.1.1.1` and `250.1.1.1`.

2. Identify the RFC 1918 addresses among: `172.20.0.1`, `172.50.0.1`, `10.9.8.7`, and `192.168.4.5`? Explain the tricky case.

3. For `192.168.1.0/26`, how many bits remain for addresses, and how many total addresses are in the block?

4. Find the subnet mask and block size for `/27`. State which octet changes for `192.168.1.0/27`.

5. List the valid `/27` subnet starting values in the fourth octet within `192.168.1.0/24`. How many child subnets fit?

6. What is the next subnet after `192.168.1.64/27`? Give its full address range.

7. How many equal-sized `/27` subnets can be created from `192.168.1.0/24`? Show borrowed bits and verify the address total.

8. How many total addresses are in `172.16.48.0/20`? Within `172.16.0.0/16`, how many `/20` subnets fit?

9. How many conventional usable host addresses does a `/28` have?

10. A point-to-point link uses a `/31`. Why is it incorrect to subtract two and conclude it has zero usable addresses?

11. How many IPv4 addresses are ordinarily assignable in an AWS `/27` subnet? Show both the total and the AWS reservation calculation.

12. Find the network range of `172.16.48.0/20`. Show why it ends where it does.

15.1 Answer key

1. `10.1.1.1` is Class A (first octet 10); `172.20.0.1` is Class B (172); `192.168.0.1` is Class C (192); `230.1.1.1` is Class D (230); and `250.1.1.1` is Class E (250). Class indicates historical classification, not whether a range is private.

2. `172.20.0.1`, `10.9.8.7`, and `192.168.4.5` are private. `172.50.0.1` is not: the private 172 range is `172.16.0.0/12`, so the second octet must be 16–31. Being outside RFC 1918 does not guarantee an address is globally routable.

3. Remaining bits: $32 - 26 = 6$. Total addresses: $2^6 = 64$.

4. A `/27` mask is `255.255.255.224`. The changing (fourth) octet has value 224, so block size is $256 - 224 = 32$.

5. Starts: 0, 32, 64, 96, 128, 160, 192, and 224. There are 8 subnets. Check: $256/32 = 8$.

6. `/27` block size is 32 in the fourth octet. $64 + 32 = 96$, so the next subnet is `192.168.1.96/27`. It contains 32 addresses and ends at `192.168.1.127`.

7. Borrowed bits: $27 - 24 = 3$. Subnets: $2^3 = 8$. Each has $2^{32-27} = 32$ addresses, and $8 \times 32 = 256$, matching the `/24` total.

8. A `/20` leaves $32 - 20 = 12$ bits, so it has $2^{12} = 4096$ addresses. From `/16` to `/20`, four bits are borrowed; $2^{20-16} = 16$ child subnets.

9. A `/28` has $2^{32-28} = 16$ total addresses. Conventional host count is $16 - 2 = 14$.

10. The traditional subtract-two rule does not apply to a `/31` point-to-point link. Under the special point-to-point convention, both of its $2^{32-31} = 2$ addresses can be used as endpoints.

11. Total: $2^{32-27} = 32$. AWS ordinarily reserves five, so $32 - 5 = 27$ addresses are assignable, subject to applicable AWS rules.

12. The mask is `255.255.240.0`, so the third-octet block size is $256 - 240 = 16$. Starting at 48, the block spans third-octet values 48 through $48 + 16 - 1 = 63$, with fourth octet 0–255. Range: `172.16.48.0`–`172.16.63.255`.

Selected official references

References

[1] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. de Groot, and E. Lear, *Address Allocation for Private Internets*, RFC 1918, February 1996. https://www.rfc-editor.org/rfc/rfc1918.

[2] *Using 31-Bit Prefixes on IPv4 Point-to-Point Links*, RFC 3021, December 2000. https://www.rfc-editor.org/rfc/rfc3021.

[3] Amazon Web Services, *Subnet CIDR Blocks*, Amazon VPC User Guide. https://docs.aws.amazon.com/vpc/latest/userguide/subnet-sizing.html.

[4] Amazon Web Services, *Add Internet Access to a Subnet*, Amazon VPC User Guide. https://docs.aws.amazon.com/vpc/latest/userguide/working-with-igw.html.

16 Final Quick Reference

Topic	Quick reference
IPv4 structure	32 bits; four 8-bit octets; each octet ranges from 0 to 255.
Historical classes	A: 0–127 by pattern (ordinary 1–126); B: 128–191; C: 192–223; D: 224–239, multicast; E: 240–255, experimental/future.
RFC 1918 private ranges	`10.0.0.0/8`; `172.16.0.0/12`; `192.168.0.0/16`.
Remaining bits	32 − prefix length.
Total addresses	$2^{32-\text{prefix length}}$.
Equal-sized subnet count	$2^{\text{new prefix}-\text{parent prefix}}$ for equal-sized child blocks.
Block size	256− mask value in the changing octet; apply only to that octet.
Conventional host count	Total minus two for traditional broadcast subnets; `/31` and `/32` are exceptions.
AWS IPv4 reservation	Ordinarily subtract five: the first four and last address are reserved. Verify applicable AWS exceptions.

IPv4 provides addresses; CIDR groups them; subnetting divides them. In an AWS VPC, good CIDR planning balances address capacity, organization, and future connectivity.